Subject outline

Lecture topics (by week)

1 – Introduction to quantum computing and maths basics
2 – Single qubit representations and logic operations
3 – Two qubit states and logic gates
4 – Multi-qubit states and quantum arithmetic
5 – Basic quantum algorithms
6 – Period finding, cryptography and quantum factoring
7 – Shor's algorithm, post-quantum crypto, quantum key distribution
8 – Quantum search algorithms
9 – Grover search applications, optimisation problems
10 – Solving optimisation problems on quantum computers
11 – Applications in quantum machine learning
12 – Real quantum computer devices

Assignment schedule:
#1: Hand out in Week 2
#2: Hand out in Week 8

Week 5

Lecture 9

9.1 Algorithms
9.2 Example Classical Algorithms and Complexity Analysis
9.3 Quantum Algorithms

Lecture 10
10.1 Deutsch-Jozsa Algorithm
10.2 Simon's Algorithm

Practice class 5

Simple Quantum Algorithms

Subject outline

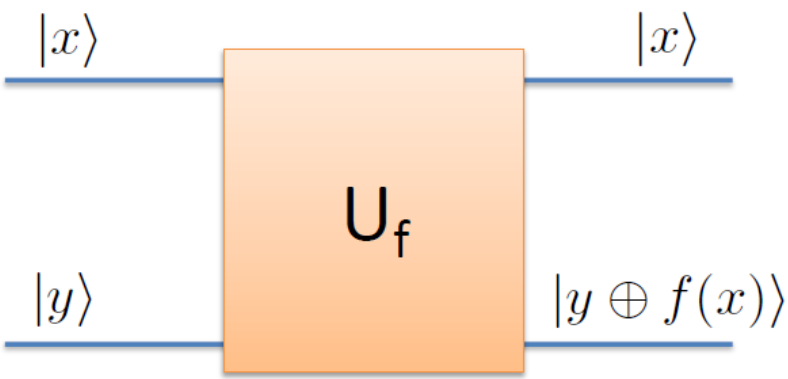
Lecture topics (by week)

1 – Introduction to quantum computing and maths basics
2 – Single qubit representations and logic operations
3 – Two qubit states and logic gates
4 – Multi-qubit states and quantum arithmetic
5 – Basic quantum algorithms
6 – Period finding, cryptography and quantum factoring
7 – Shor's algorithm, post-quantum crypto, quantum key distribution
8 – Quantum search algorithms
9 – Grover search applications, optimisation problems
10 – Solving optimisation problems on quantum computers
11 – Applications in quantum machine learning
12 – Real quantum computer devices

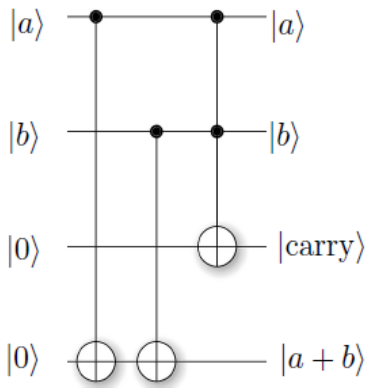
Assignment schedule:
#1: Hand out in Week 2
#2: Hand out in Week 8

Recap: Lecture 8

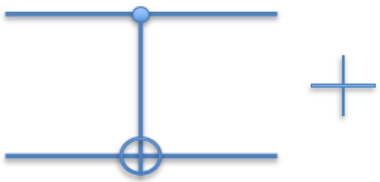
Reversible functions:
Implementing *f(x)* on a quantum computer.



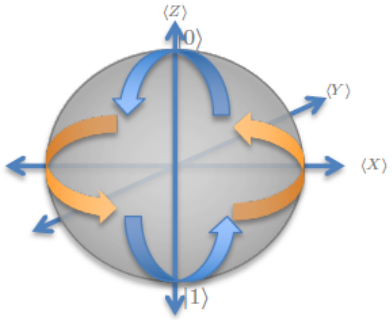
Arithmetic:
One qubit adder on a quantum computer.



Universality:
In quantum computing every algorithm can be expressed using just CNOT and single qubit rotations



CNOT



Single Qubit Rotations

9.1 Algorithms

MULT20015

Lecture 9

What is an algorithm?

An algorithm is a well-defined systematic set of rules or instructions that specifies a series of operations to be performed on a data (often called input), producing after a finite *amount of time* some output data.

A few salient features of an algorithm are:

- It leads to a solution of a computational problem
- It can be readily translated into a computer program
- The *amount of time* required to execute it is its cost, represented by a well-defined function

Algorithm Design

The efficient design of an algorithm should consider:

- Problem to be solved
- What are the existing solutions? Can we do better?
- Are there multiple solutions?
- What is the cost in terms of number of steps (time), memory?

Algorithm Analysis

The analysis of an algorithm involves:

- Does it solve the problem?
- Does it solve the problem for all possible inputs?
- What is the its cost?
- What is the best case, average case, worst case costs?

9.2 Example Classical Algorithms and Complexity Analysis

MULT20015

Lecture 9

Example 1: Search in an ordered array

Input Data: A sorted array of 11 integers: (n=11)

Example: {2, 4, 6, 9, 11, 34, 41, 65, 87, 90, 98}

Output Data: Find the index of a particular number m in the list. Suppose m=87?

Algorithm: (Binary Search)

1) Find middle entry of input data

2) Compare target with the middle entry:

if target = middle entry, output data found. Exit.

if target > middle entry, input data = upper half of input data. Repeat 1 & 2.

if target < middle entry, input data = lower half of input data. Repeat 1 & 2.

Example 1: Search in an ordered array

Index	0	1	2	3	4	5	6	7	8	9	10
Input Data =	2	4	6	9	11	34	41	65	87	90	98

Middle = 0+[(10-0)/2] = 5

Input Data =	2	4	6	9	11	34	41	65	87	90	98

87 > 34 ➔ **Input Data**

Index	6	7	8	9	10
Input Data =	41	65	87	90	98

Middle = 6+[(10-6)/2] = 8

Input Data =	41	65	87	90	98

87 = 87 ➔ Output Data = 8

Calculation of middle index:

Middle = Lowest index + [(Highest index – Lowest Index)]/2

Complexity or Cost of an Algorithm, Big-O and Big-Omega

Cost or complexity of an algorithm is concerned with how fast or slow an algorithm executes for a given input set. Typically, it is defined as T_n, the time to run the algorithm for an input size of n.

T_n not only depends on algorithm design, but also on other parameters such as

- Memory required vs Memory available
- Processor speed
- Instruction set
- Compiler efficiency

Same algorithm may execute in drastically different time on different hardware platforms

Complexity or Cost of an Algorithm, Big-O and Big-Omega

One way to get around hardware dependence, algorithm cost or complexity can be defined asymptotically i.e. in terms of "elementary steps"

For example:

Addition of two 4-bit numbers (such as 1000 and 1010) would require 4 elementary steps i.e. addition of each bit one by one. [Note: carry is ignored here which will be additional overhead]

Here n = 4, and the number of elementary steps ("addition of two bits") are also 4

Therefore, $T_4 = 4$

And more generally,

$T_n = n$

Note: the actual execution time of the algorithm would be $T_n = C_e * n$, where C_e is the cost of one elementary step based on a specific hardware platform.

Complexity or Cost of an Algorithm, Big-O and Big-Omega

Asymptotic Cost Notations:

The time to execute an algorithm is T_n, however, often the cost function f(n) is used to define the efficiency of an algorithm: *i.e.* $T_n = f(n)$ for a given value of n [Note: C_e is ignored]

The algorithms are classified in groups in accordance with their asymptotic cost functions

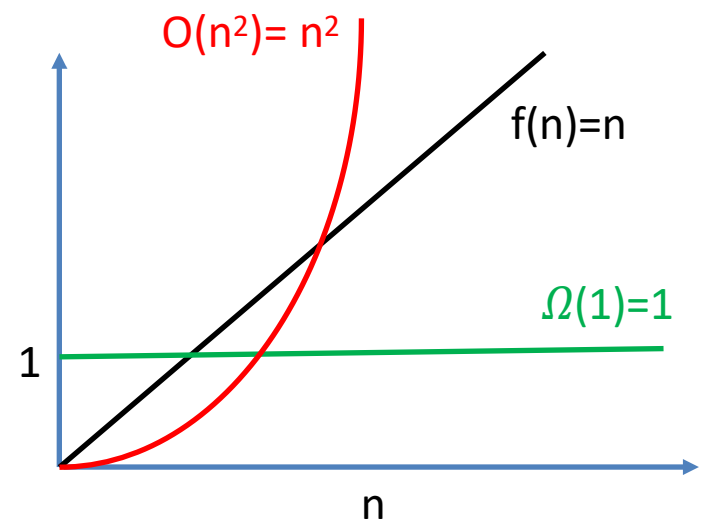
An asymptotic cost function defines an upper or lower bound to the actual cost function f(n)

Example: An algorithm with cost function f(n) = n can be plotted as a linear curve with respect to n.

For a reasonably large value of n:

$O(n^2)$ will always be greater than f(n)
$\Omega(1)$ will always be smaller than f(n)

Therefore, $O(n^2)$ defines an upper bound to f(n), and $\Omega(1)$ defines a lower bound to f(n).

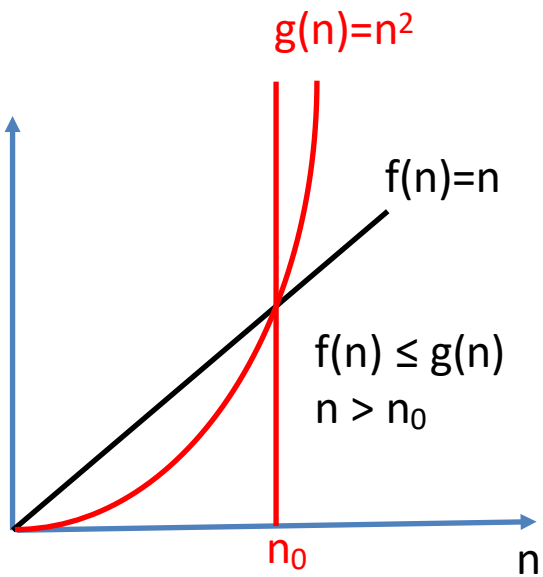


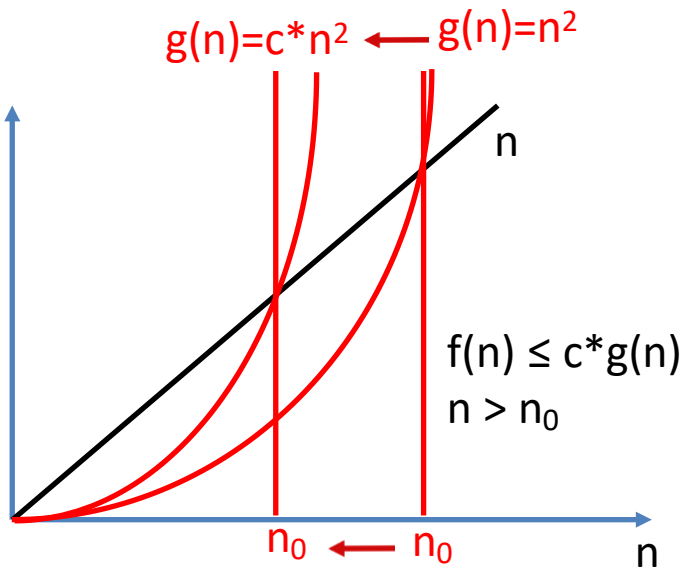
Formal definition of Big-O

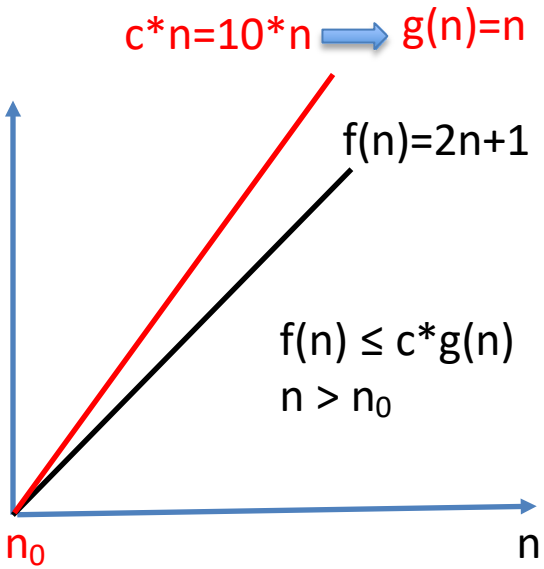
For any monotonic functions f(n) and g(n) from the positive integers to the positive integers, we say that $f(n) = O(g(n))$ when there exist constants $c > 0$ and $n_0 > 0$ such that $f(n) \leq c * g(n)$, for all $n \geq n_0$

Intuitively, this means that function f(n) does not grow faster than g(n), or that function g(n) is an **upper bound** for f(n), for all sufficiently large $n \to \infty$

Examples: $1 = O(n)$
$\log n = O(n)$
$n = O(n^2)$
$2n+1 = O(n)$







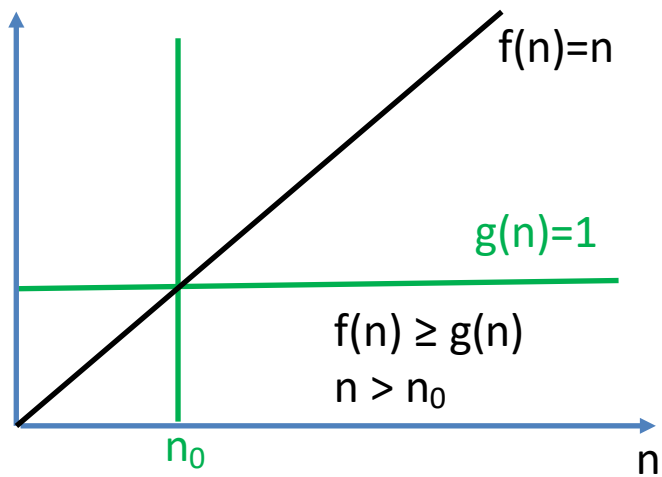
Formal definition of Big-Omega

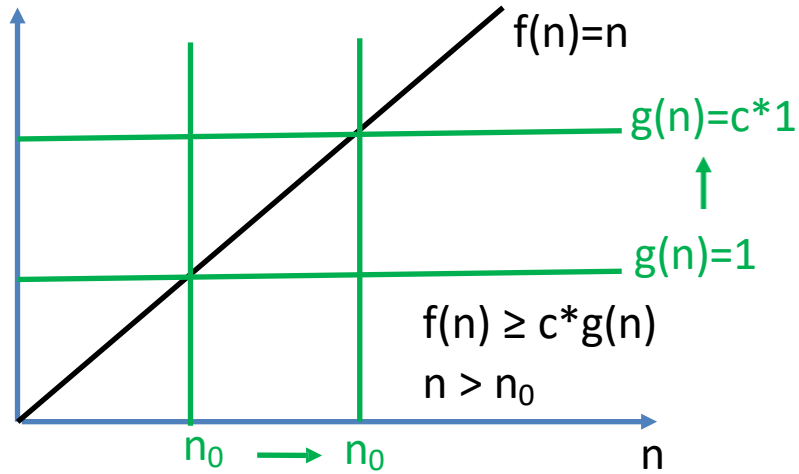
For any monotonic functions f(n) and g(n) from the positive integers to the positive integers, we say that $f(n) = \Omega(g(n))$ when there exist constants $c > 0$ and $n_0 > 0$ such that $f(n) \geq c*g(n)$, for all $n \geq n_0$

Intuitively, this means that function g(n) is a **lower bound** for f(n), for all sufficiently large $n \to \infty$

Examples:

$n = \Omega(1)$
$n^2 = \Omega(n)$
$n^2 = \Omega(n \log n)$





Complexity of binary search algorithm

Recall Binary Search Algorithm:

1) Find middle entry of input data

2) Compare target with the middle entry:

if target = middle entry, output data found. Exit.

if target > middle entry, input data = upper half of input data. Repeat 1 & 2.

if target < middle entry, input data = lower half of input data. Repeat 1 & 2.

Elementary Step: Comparison of target number with the middle entry

Each elementary step divides the data set in half and under worst case, the comparison will be done until the input size reduces to just a single element

Therefore, number of elementary steps required for input size n would be $\lfloor \log_2 n + 1 \rfloor$

Hence f(n) = $\lfloor \log_2 n + 1 \rfloor$

Asymptotic costs: $\lfloor \log_2 n + 1 \rfloor$ = $O(\log_2 n)$ and $\lfloor \log_2 n + 1 \rfloor$ = $\Omega(1)$

Floor function:
$\lfloor 3.9 \rfloor = 3$
$\lfloor 4.3 \rfloor = 4$
$\lfloor 5 \rfloor = 5$

Example 2: Sorting an array of disordered data

Input Data: An array of 11 unsorted integers: (n=11)

Example: {2, 9, 6, 10, 16, 24, 49, 37, 87, 51, 41}

Output Data: A new array consisting of sorted Input Data

Sorted array = All numbers are arranged in the array in increasing or decreasing order.

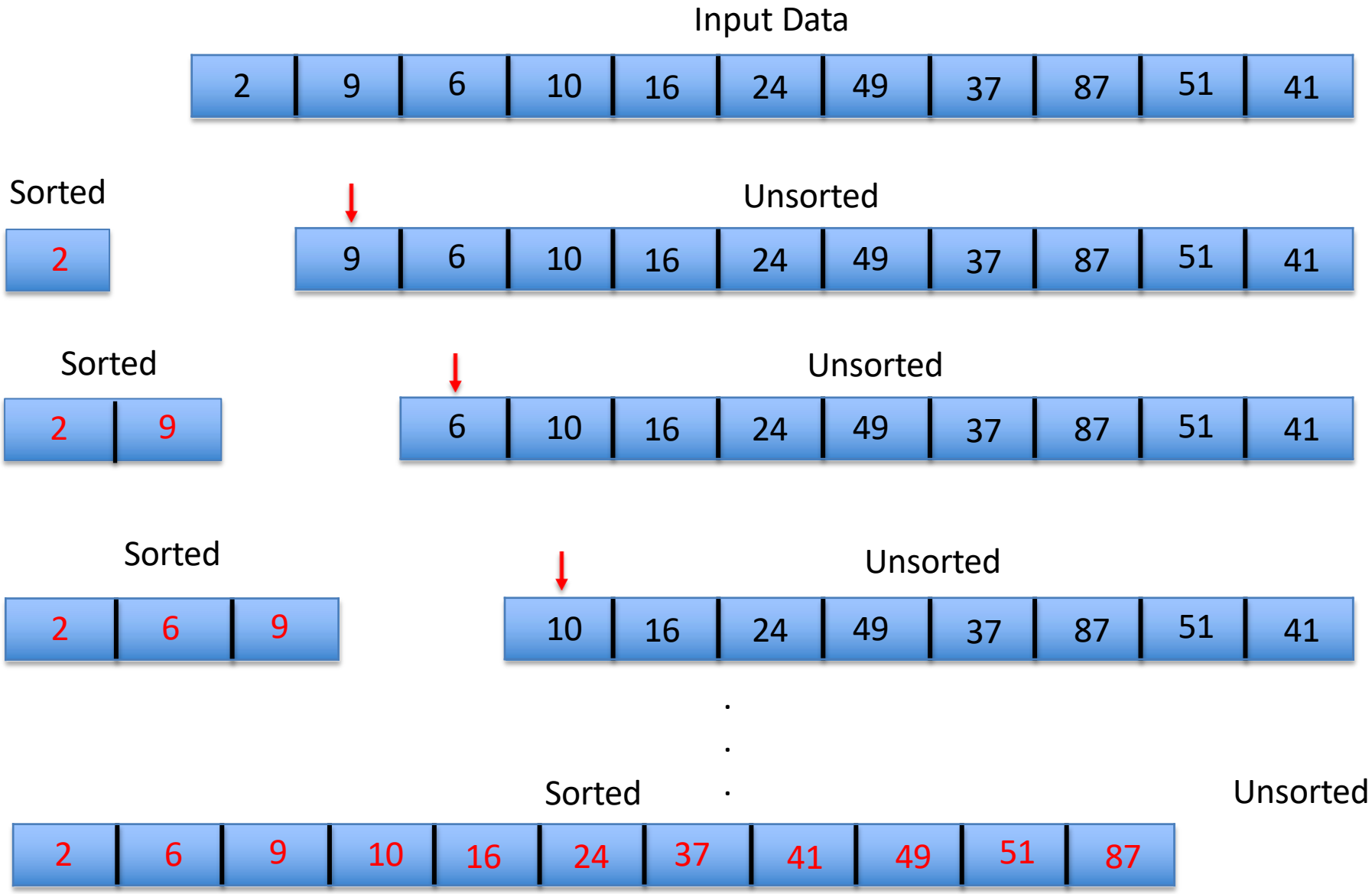
Algorithm: (Insertion Sort)

1) Initialize: sorted array = first element of Input Data, unsorted array = rest of the Input Data

2)Remove first element from the unsorted array and place it at its correct sorted location in the sorted array by linear comparisons.

3) Repeat until all elements from the unsorted array are removed.

Example 2: Sorting an array of disordered data



Complexity of Insertion Sort Algorithm

Algorithm: (Insertion Sort)

1) Initialize: sorted array = first element of Input Data, unsorted array = rest of the Input Data

2)Remove first element from the unsorted array and place it at its correct sorted location in the sorted array by linear comparisons.

3) Repeat until all elements from the unsorted array are removed.

Elementary Step: Comparison of elements from unsorted array with elements of sorted array.

Number of comparison	Size of sorted array
0	0
0	1
1	2
2	3
.	.
.	.
n-2	n-1
n-1	n

Total number of elementary steps for size n:

$$f(n)= 0+1+2+3+\ldots+(n-2)+(n-1) = \frac{n(n-1)}{2}$$

Asymptotic costs:

$$\frac{n(n-1)}{2} = O(n^2)$$

$$\frac{n(n-1)}{2} = \Omega(n)$$

Example 2: Greatest Common Divisor (GCD)*

Input Data: Two non-negative, not both 0, integers m and n

Example: m=10, n=4

Output Data: Largest integer that divides both m and n.

Algorithm: (Euclid Algorithm)

1) Check if m and n both are 0; Yes, exit; No, go to next step.

2) If n is not zero, perform r = m mod n; copy n into m; copy r into n.

3) Repeat step 2 while n is not zero. When n is zero, m is the answer.

*Highly relevant for Shor's Quantum Factoring Algorithm

Example 3: Greatest Common Divisor (GCD)

m=10, n=4

n is not zero: r = 10 mod 4 = 2

Copy n into m: m = 4 Copy r into n: n = 2

n is not zero: r = 4 mod 2 = 0

Copy n into m: m = 2 Copy r into n: 0

n = 0: Answer=m: 2

Definition of "mod" operation:

"m" mod "n": Remainder of "m"

Examples:

10 mod 4 = 2
4 mod 2 = 0
6 mod 5 = 1

Elementary Step: calculation of mod function between two integers

Home reading: Find the complexity class for GCD Algorithm.

Note: Convince yourself that if we assume "mod" operation takes unit time to run, then the complexity class for the GCD algorithm is O(log p), where p is the largest of m and n.

9.3 Quantum Algorithms

MULT20015

Lecture 9

Quantum Algorithms

A quantum algorithm is an algorithm which is designed to solve a computational problem on a quantum computer.

Obvious question:

Why can't we simply run a classical algorithm on a quantum computer and perform the computational task much faster?

Answer:

1) Quantum computers are fundamentally different from classical computers and therefore algorithms need to be designed to exploit quantum properties such as superposition, entanglement, etc.

2) Hypothetically, if classical algorithm is to run on a quantum computer, it might only improve C_e, not necessarily f(n).

Quantum Algorithms

So, how can superposition and entanglement can help to speed up the computation?

Superposition can prepare a quantum state that spans the whole Hilbert space

-> quantum state could encompass an exponentially large data space

Entanglement couples the qubits such that common operations can be performed

-> operations can collectively enhance/suppress the amplitude probabilities

Often you would see that quantum algorithms follow a general recipe or structure:

-> prepare a superposition quantum state := all possible outcomes with equal probability
-> apply quantum operations := enhance probability of desired state
reduce probability of other outcomes
-> do the measurement := collapse the quantum state to desired output

Existing Quantum Algorithms

Compared to the field of classical algorithms, quantum algorithm discipline is relatively new and a topic of highly active research.

There are a 'few hundred' quantum algorithms already exist (http://quantumalgorithmzoo.org/)

Major Categories:

Algebraic/Number Theoretic:

-> Factoring [Week 7]
-> Discrete log
-> Verifying matrix products

Search/database:

-> Grover search [Week 8]
-> Amplitude amplification

Variational/Simulation:

-> Variational Quantum Eigensolver
-> Quantum Approximate Optimisation [Week 10]

Machine Learning:

-> Quantum Principal Component Analysis
-> Quantum Support Vector Machines [Week 11]
-> Quantum Neural Network

9.4 Algorithms

MULT20015

Lecture 9

Week 5

Lecture 9

9.1 Algorithms
9.2 Example Classical Algorithms and Complexity Analysis
9.3 Quantum Algorithms

Lecture 10
10.1 Deutsch-Jozsa Algorithm
10.2 Simon's Algorithm

Practice class 5

Simple Quantum Algorithms

Subject outline

Lecture topics (by week)

1 – Introduction to quantum computing and maths basics
2 – Single qubit representations and logic operations
3 – Two qubit states and logic gates
4 – Multi-qubit states and quantum arithmetic
5 – Basic quantum algorithms
6 – Period finding, cryptography and quantum factoring
7 – Shor's algorithm, post-quantum crypto, quantum key distribution
8 – Quantum search algorithms
9 – Grover search applications, optimisation problems
10 – Solving optimisation problems on quantum computers
11 – Applications in quantum machine learning
12 – Real quantum computer devices

Assignment schedule:
#1: Hand out in Week 2
#2: Hand out in Week 8